

HealthGuard AI Interview Guide

This document explains HealthGuard AI from a senior software engineering interview perspective. It covers architecture, major features, request flows, RAG/LLM design, database design, security decisions, trade-offs, and likely cross-questions.

1. One-Minute Project Summary

HealthGuard AI is a preventive health-risk assistant. It collects patient lifestyle, diet, symptoms, climate, occupation, and optional uploaded health documents, then generates an educational, safety-gated health risk report. The system uses rule-based risk scoring plus Gemini-based report enhancement. It includes authentication, email confirmation, Google SSO support, role-based dashboards, doctor review workflows, PDF report download, audit logging, and a local RAG pipeline for patient documents and internal knowledge.

The product is not a diagnostic tool. It explicitly avoids diagnosis, prescriptions, medication changes, and emergency decision-making. It provides educational risk awareness and doctor-consultation guidance.

2. High-Level Architecture

The application is a FastAPI backend with a static frontend.

Main layers:

- Frontend: frontend/index.html, frontend/app.js, frontend/styles.css
- API layer: backend/app/api/routes.py
- Domain services: backend/app/services/*
- Schemas: backend/app/schemas/health.py
- Database abstraction: backend/app/db/store.py
- Tests: backend/tests/test_safety_acceptance.py
- Tracking docs: docs/TRACK.md

Runtime flow:

```
Browser UI
-> FastAPI route
-> Auth/session validation
-> Domain service
-> Database/RAG/LLM/PDF/email service as needed
-> JSON or PDF response
-> Frontend renders result
```

3. Core Functionalities

Authentication And Registration

Implemented:

- User registration
- Login
- Email verification using 6-digit code
- SMTP-based confirmation email
- Session token stored in browser local storage

- Role-based capabilities
- Google SSO OAuth start/callback support

Roles:

- Patient
- Doctor
- Dietician
- Admin
- Compliance

Important behavior:

- Users cannot access dashboard features before login.
- Users cannot log in until email is verified.
- Failed registration does not leave a completed user account.
- Google SSO creates or updates a verified user from Google account info.

Interview answer:

?Authentication is implemented through demo session tokens and verified email gating. Registration creates a pending user only after email delivery is available. Login requires email verification. For Google SSO, the OAuth callback exchanges the code with Google, fetches userinfo, validates verified email, upserts the local user, and creates the same local demo session used by normal login.?

Likely cross-question: Why still use app sessions after Google SSO-

Answer:

?Google proves identity, but our app still needs an internal session and role/capability model. After OAuth, we map the Google user to a local user and issue our own session token so all downstream authorization is consistent.?

4. Role-Based Access

Patients:

- Create health reports
- Upload documents
- View only their own saved reports
- Download only their own reports

Doctors/Dieticians:

- View pending reports
- Review and approve reports
- See pending reports grouped by patient folder

Admins:

- Add approved knowledge
- Manage internal guidance

Compliance:

- View audit logs

Important privacy decision:

Patient reports are isolated by `patient_profiles.user_id`. The report list, detail, and

PDF download endpoints enforce ownership. Older reports created before ownership migration may have user_id = null, but new reports are bound to the logged-in patient.

Interview answer:

?We enforce data isolation at the backend, not just in the UI. The report query joins reports to patient profiles and filters by the logged-in user ID for patient users.

Doctor/dietician/admin/compliance roles use elevated access paths. The frontend hiding is only UX; security is enforced in API routes and storage queries.?

5. Patient Assessment And Report Flow

Frontend collects:

- Age
- Gender
- Height/weight
- Location
- Climate/environment
- Occupation
- Sleep hours
- Exercise level
- Diet pattern
- Food catalog selections
- Water intake
- Smoking/alcohol
- Existing conditions
- Family history
- Current medications
- Allergies
- Symptoms
- Main question
- Consent

Backend route:

POST /api/reports/generate

Flow:

```
Validate session
-> Validate consent
-> Validate mandatory fields via Pydantic
-> Triage symptoms
-> Score risk
-> Generate rule-based precautions
-> Retrieve RAG context
-> Build base report
-> Call Gemini
-> Merge Gemini enhancement
-> Save assessment and report
-> Save audit log
-> Return report JSON
```

Important decision:

If Gemini is configured but fails, report generation stops with HTTP 503. The system does not silently save a rules-only report because the requirement was to make LLM generation

functional and transparent.

Interview answer:

?The report is not purely LLM-generated. We first build a deterministic safety report using triage, risk scoring, and guidance services. Gemini then enhances it with educational language and extra safe questions/reminders using RAG context. This reduces hallucination risk and gives us predictable safety boundaries.?

6. Risk And Safety Design

Rule-based components:

- TriageService: detects emergency red flags.
- RiskService: calculates risk score and level.
- GuidanceService: creates lifestyle, diet, and climate precautions.
- SafetyService: blocks unsafe medication-prescribing behavior in chat.

Why rule-based first-

- Medical safety needs deterministic boundaries.
- LLM output can vary.
- Red flags should not depend on LLM interpretation.
- Risk score should be explainable.

Interview answer:

?For a health product, I would not rely on an LLM for primary safety classification. We use deterministic red-flag and risk scoring logic first, then use the LLM only as an enhancement layer.?

7. LLM Integration

Provider:

- Gemini

Model:

- Primary: gemini-2.5-flash
- Fallback: gemini-2.5-flash-lite

Endpoint:

```
models/{model}:generateContent
```

LLM behavior:

- JSON response schema requested
- Safety instructions included
- No diagnosis
- No prescriptions
- No medication changes
- Escalate emergency symptoms
- Use uploaded context only as supporting user-provided data

Reliability:

- Retry logic for transient 429/5xx/network errors
- Fallback model support

- Clear HTTP 503 if all attempts fail

Interview answer:

?Gemini is used through the standard generateContent REST endpoint with structured JSON output. We retry transient failures and fall back to a lighter model. If all configured models fail, the backend returns a 503 and does not persist a report.?

Likely cross-question: Why not let rules fallback generate the report-

Answer:

?For this requirement, the user explicitly wanted LLM-functional reports and no misleading output when LLM fails. So we fail closed: no report is saved unless the LLM enhancement succeeds.?

8. RAG Pipeline

Current RAG implementation is complete for local/demo use.

Components:

- DocumentService: extracts and cleans file text.
- RAGService: chunks, embeds, stores, and retrieves semantic chunks.
- KnowledgeService: retrieves global and patient-specific context.
- ReportService and ChatService: pass retrieved context to Gemini.

Storage:

- rag_documents
- rag_chunks

Embedding provider:

- Local deterministic hashing embedding

Vector store:

- Existing app database with JSON vectors in rag_chunks.embedding_json

Similarity:

- Cosine similarity in Python

Flow:

```
User uploads file
-> Extract text
-> Clean text
-> Chunk text
-> Generate local embedding per chunk
-> Store metadata in rag_documents
-> Store chunks/vectors in rag_chunks
-> On report/chat, embed query
-> Retrieve matching global + patient-owned chunks
-> Rank by cosine similarity
-> Send top chunks to Gemini
```

Data isolation:

- Patient-uploaded chunks are stored with user_id.
- Retrieval uses:

```
rag_documents.user_id IS NULL OR rag_documents.user_id = current_user_id
```

So global knowledge is shared, but patient documents are private.

Interview answer:

?RAG is implemented as a local semantic retrieval pipeline. Uploaded documents are extracted, cleaned, chunked with overlap, embedded using deterministic local embeddings, and stored in database-backed vector tables. At generation time, we embed the query, run cosine similarity over candidate chunks, retrieve top-ranked chunks, and pass them to Gemini as context. Patient uploads are isolated by user ID, while admin-approved knowledge is global.?

Likely cross-question: Which vector database are you using-

Answer:

?Currently we use the app database as a lightweight vector store, storing embeddings as JSON and ranking in Python. This is suitable for local/demo use. For production, I would migrate to pgvector or a dedicated vector DB like Pinecone, Chroma, or FAISS for indexed ANN search.?

Likely cross-question: Why local embeddings-

Answer:

?It avoids another external dependency and makes the demo deterministic and runnable offline. For production, I would use a stronger medical/general embedding model and store vectors in pgvector.?

9. File Extraction Flow

Supported:

- Text-like files: .txt, .md, .csv, .json
- DOCX
- Basic text PDFs

Current extraction:

- Text files: decoded with UTF-8, UTF-16, cp1252, latin-1 fallback.
- DOCX: reads word/document.xml from ZIP.
- PDF: basic PDF text operator extraction.

Not yet production-grade:

- OCR for scanned PDFs/images
- Table extraction
- Lab-value normalization
- Extraction confidence score

Interview answer:

?The current file extraction supports text, DOCX, and basic text PDFs. After extraction, we normalize whitespace, redact secret-like values, summarize for UI, then index the cleaned text into RAG. For production medical documents, I would add OCR, structured table extraction, lab-value normalization, and extraction confidence.?

10. Database Design

Main tables:

- users
- sessions
- email_outbox
- patient_profiles
- assessments
- reports
- knowledge_documents
- rag_documents
- rag_chunks
- audit_logs

Important relationships:

```
users.id -> sessions.user_id
users.id -> patient_profiles.user_id
patient_profiles.id -> assessments.patient_id
patient_profiles.id -> reports.patient_id
rag_documents.user_id -> users.id
rag_chunks.document_id -> rag_documents.id
```

Database support:

- SQLite fallback
- PostgreSQL via DATABASE_URL

Interview answer:

?The app is database-agnostic through a small adapter layer. It supports SQLite for local/demo and PostgreSQL through DATABASE_URL. For production, I would use PostgreSQL with migrations and pgvector.?

11. Report Persistence And PDF

After successful LLM-enhanced report:

- assessment input is saved
- report JSON is saved
- audit log is saved
- PDF download can be generated

PDF route:

```
GET /api/reports/{report_id}/download
```

Access:

- Patient can download only own report.
- Doctor/dietician/admin/compliance can access review queues.

Interview answer:

?We store the report as JSON, which keeps the generated output structured and auditable. PDF generation is derived from stored report data, not regenerated from the LLM.?

12. Audit Logging

Tracked actions include:

- registration
- login
- email verification
- report generation
- document upload
- doctor review
- admin knowledge creation
- SSO login

Why:

- traceability
- compliance visibility
- debugging
- safety review

Interview answer:

?For health-related applications, auditability matters. We record key user actions and report outcomes so a compliance/admin role can inspect activity.?

13. Frontend Architecture

Frontend is static HTML/CSS/JS.

Key behaviors:

- Auth view before login
- Dashboard after verified login
- Role tools hidden for patients
- Report output shown at bottom after generation
- Saved reports paginated
- Upload documents optional
- Google/Facebook/Instagram SSO buttons
- Google SSO live path implemented

Interview answer:

?The frontend is intentionally lightweight. Most security decisions are enforced on the backend; frontend role hiding is only for UX.?

14. Deployment And Public URL

Local server:

```
python -m uvicorn backend.app.main:app --host 127.0.0.1 --port 8001
```

Public temporary URL:

- ngrok tunnel to local app

Important:

- For Google SSO through ngrok, Google redirect URI must use the ngrok domain.
- For local SSO, redirect URI must be:

```
http://127.0.0.1:8001/api/auth/sso/google/callback
```


15. Testing Strategy

Test file:

```
backend/tests/test_safety_acceptance.py
```

Coverage includes:

- registration
- email confirmation
- login
- mandatory assessment fields
- report generation
- LLM test mode
- document upload
- RAG indexing/retrieval
- patient RAG isolation
- patient report privacy
- doctor review
- admin knowledge
- role guards
- SSO start scaffold

Current suite:

```
20 tests passing
```

Interview answer:

?The tests are acceptance-style and validate end-to-end behaviors: auth, role guards, report generation, RAG retrieval, and patient privacy. I added regression tests specifically for patient data isolation and RAG isolation.?

16. Security And Privacy Decisions

Implemented:

- Session required for feature APIs
- Email verification required
- Patient report filtering
- Patient document RAG isolation
- Role-based access checks
- No password hash exposure
- No secret printing
- Uploaded text secret-like redaction
- No report saved when LLM fails

Important principle:

Security is enforced in backend routes/storage, not only frontend visibility.

17. Known Limitations

Current limitations:

- Demo session tokens, not production JWT/OIDC session management

- No database migrations framework yet
- SQLite fallback is not production-grade
- RAG vectors stored as JSON, not pgvector
- Local hashing embeddings are not medical-grade
- PDF extraction is basic
- No OCR for images/scanned PDFs
- No structured lab parser
- No background job queue
- No rate limiting
- Google SSO implemented; Facebook/Instagram still start-scaffold only

18. Production Improvements

If asked what I would improve:

- PostgreSQL with Alembic migrations
- pgvector for indexed vector search
- production embeddings
- OCR pipeline for scans/images
- structured medical entity extraction
- encrypted PHI storage
- audit trails with user IDs everywhere
- proper OIDC/session management
- background workers for file processing
- virus scanning for uploads
- rate limiting and abuse protection
- observability with logs/metrics/traces
- deployment behind HTTPS reverse proxy
- secrets manager instead of .env

19. Strong Cross-Question Answers

Q: Is this a medical diagnosis system-

No. It is an educational preventive health-risk assistant. It does not diagnose, prescribe, or change medication. It flags risk factors and encourages doctor consultation.

Q: Why combine rules and LLM-

Rules give deterministic safety boundaries. LLM gives natural-language enhancement. In healthcare, I would not let the LLM be the only decision maker.

Q: How do you prevent patient data leakage-

Reports are tied to patient_profiles.user_id, and RAG documents are tied to rag_documents.user_id. Patient APIs filter by the logged-in user. Doctor/admin roles have explicit elevated paths.

Q: What happens if Gemini fails-

The backend returns HTTP 503 and does not save the report. This prevents a user from thinking they received an LLM-generated report when they did not.

Q: What data is sent to Gemini-

Only selected structured patient inputs, rule-based report summary, and top retrieved RAG chunks. The full raw uploaded file is not sent.

Q: How does RAG retrieval work-

The query is embedded, candidate chunks are loaded from global and user-owned RAG documents, cosine similarity is calculated, chunks are ranked, and the top sources are passed to Gemini.

Q: Why not use Pinecone or pgvector-

For local demo portability, embeddings are stored as JSON and ranked in Python. For production, pgvector would be my first choice because the app already supports PostgreSQL.

Q: How would you handle scanned lab reports-

Add OCR, image preprocessing, table extraction, lab-value normalization, and confidence scoring. Low-confidence extraction should go to manual review.

Q: How do doctors see patient reports-

Doctors see pending reports grouped into patient folders. Patients see only their own saved reports.

Q: What is the most important engineering decision-

Failing closed for safety: no dashboard without verified login, no cross-patient report access, no report saved when LLM fails, and no diagnosis/prescription behavior.

20. Interview Closing Summary

?This project is designed as a safety-first health AI assistant. The backend owns the trust boundaries: authentication, email verification, role access, patient isolation, RAG retrieval, deterministic risk scoring, and LLM failure behavior. The LLM is used as an enhancement layer, not as the primary safety engine. The RAG pipeline persists user documents, chunks and embeds them, retrieves relevant patient-specific and global context, and feeds only the top context into Gemini. The current implementation is demo-ready and locally portable, while the production path is clear: PostgreSQL, pgvector, OCR, stronger embeddings, migrations, and hardened security controls.?